

Assignment 1

-Graphs-

Stud: Balaci Klara-Sofia

Group:911/1

Date:03.04.2025

I. Class DirectedGraph

First of all, I defined a class named DirectedGraph(in python) and Graph(in c++), both representing a directed graph. We now take a look at the constructor of these classes.

Python source code:

```
def __init__(self, v=None, in_edges=None, out_edges=None, costs=None):
if in_edges is None and out_edges is None and costs is None:
    self._out_edges = {}
    self._in_edges = {}
    self._costs = {}
    if v is None:
        v = []
    if isinstance(v, int):
        for i in range(v):
            self._out_edges[i] = []
            self._in_edges[i] = []
    else:
        for i in v:
            self._out_edges[i] = []
            self._in_edges[i] = []
    else:
        self._out_edges = out_edges
        self._in_edges = in_edges
        self._costs = costs
```

The class consists of 3 dictionaries: in_edges,out_edges(the key represents a vertex and it s value is a list containing all it s inbound/outbound edges) and costs(the key is a tuple representing an edge between the two values and the pair value represents the cost)

C++ source code:

```
class Graph {
private:
    std::map<int, std::vector<int>>> d_in;
    std::map<int, std::vector<int>>> d_out;
    std::map<std::pair<int, int>, int> cost;
public:
    Graph() = default;
    Graph(int v);
    Graph(std::vector<int> v);
    ~Graph() = default;
```

The structure is identical to the one in python but we have defined 4 constructors:

->the default one

->Graph::Graph(int v) {

//in case v represents the number of vertices

```
    for (int i = 0; i < v; i++) {
        d_in[i] = std::vector<int>();
        d_out[i] = std::vector<int>();
    }
}
```

->Graph::Graph(std::vector<int> v) {

//in case v is an iterable that holds all the vertices

```
    for (auto node : v) {
        d_in[node] = std::vector<int>();
        d_out[node] = std::vector<int>();
    }
}
```

->Graph::Graph(std::map<int, std::vector<int>>> in, std::map<int, std::vector<int>>> out, std::map<std::pair<int, int>, int> costs) {

```
    d_in = in;
    d_out = out;
```

```
    cost = costs;
}
```

Further I defined the following methods:

1.Number_of_vertices//returns the number of vertices in the graph

Python source code:

```
def number_of_vertices(self):
    return len(self._in_edges.keys())
```

C++ source code:

```
int Graph::Number_of_Vertices() {
    return int(this->d_in.size());
}
```

2.Parse_vertices//returns an inerrable containing all the vertices in the graph

Python source code:

```
def parse_vertices(self):
    return self._out_edges.keys()
```

C++ source code:

```
std::vector<int> Graph::Parse_Vertices() {
    std::vector<int> vertices;
    for (const auto& pair : d_out) {
        vertices.push_back(pair.first);
    }
    return vertices;
}
```

3. Is_edge//returns true if there exists an edge between two given vertices false otherwise; throws exception if one of the given vertices does not exist.

Python source code:

```
def is_edge(self, x, y):
    if not self.valid_vertex(x):
        raise ValueError("Vertex {} does not exist.".format(x))
    if not self.valid_vertex(y):
```

```
        raise ValueError("Vertex {} does not exist.".format(y))
    return y in self._out_edges[x]
```

C++ source code:

```
bool Graph::Is_Edge(int x, int y) {
    if (!Valid_Vertex(x) || !Valid_Vertex(y)) {
        throw std::invalid_argument("Vertex does not exist");
    }
    // We must also check if the edge is in the cost map
    return cost.find({x, y}) != cost.end();
}
```

4. InOut_degree//returns the in and out degree of a given vertex; throws an exception if the vertex does not exist

Python source code:

```
def inout_degree(self, x):
    if not self.valid_vertex(x):
        raise ValueError("Vertex {} does not exist.".format(x))
    return len(self._in_edges[x]), len(self._out_edges[x])
```

C++ source code:

```
std::pair<int, int> Graph::InOut_Degree(int x) {
    if (!Valid_Vertex(x)) {
        throw std::invalid_argument("Vertex does not exist");
    }
    return {d_in[x].size(), d_out[x].size()};
}
```

5. Parse_out//returns an iterable containing all outbound neighbors of a given vertex; throws an exception if the vertex does not exist

Python source code:

```
def parse_out(self, x):
    if not self.valid_vertex(x):
```

```
        raise ValueError("Vertex {} does not exist.".format(x))
    return list(self._out_edges[x])
```

C++ source code:

```
std::vector<int> Graph::Parse_Out(int x) {
    if (!Valid_Vertex(x)) {
        throw std::invalid_argument("Vertex does not exist");
    }
    return d_out[x];
}
```

6. Parse_in//returns an iterable containing all inbound neighbors of a given vertex; throws an exception if the vertex does not exist

Python source code:

```
def parse_in(self, y):
    if not self.valid_vertex(y):
        raise ValueError("Vertex {} does not exist.".format(y))
    for x in self._in_edges[y]:
        yield x
```

C++ source code:

```
std::vector<int> Graph::Parse_In(int x) {
    if (!Valid_Vertex(x)) {
        throw std::invalid_argument("Vertex does not exist");
    }
    return d_in[x];
}
```

7. Get_cost//returns the cost of a given edge; throws an exception if one or neither one of the vertices does not exist or if the edge between the given vertices does not exist

Python source code:

```
def get_cost(self, v1, v2):
    if not self.valid_vertex(v1):
        raise ValueError('vertex {} not in graph'.format(v1))
    if not self.valid_vertex(v2):
        raise ValueError('vertex {} not in graph'.format(v2))
```

```

if not self.is_edge(v1, v2):
    raise ValueError(f'Edge ({v1}, {v2}) not in graph')
return self._costs[(v1, v2)]

```

C++ source code:

```

int Graph::Get_Cost(int v1, int v2) {
    if (!Valid_Vertex(v1) || !Valid_Vertex(v2)) {
        throw std::invalid_argument("One or both vertices do not exist");
    }
    if (!Is_Edge(v1, v2)) {
        throw std::invalid_argument("Edge does not exist");
    }
    return cost[{v1, v2}];
}

```

8. Update_cost//modifies the cost of an edge; throws an exception if one or neither one of the vertices do not exist or if the edge between the given vertices does not exist

Python source code:

```

def update_cost(self, v1, v2, cost):
    if not self.valid_vertex(v1):
        raise ValueError('vertex {} not in graph'.format(v1))
    if not self.valid_vertex(v2):
        raise ValueError('vertex {} not in graph'.format(v2))
    if not self.is_edge(v1, v2):
        raise ValueError(f'Edge ({v1}, {v2}) not in graph')
    self._costs[(v1, v2)] = cost

```

C++ source code:

```

void Graph::Update_Cost(int v1, int v2, int new_cost) {
    if (!Valid_Vertex(v1) || !Valid_Vertex(v2)) {
        throw std::invalid_argument("One or both vertices do not exist");
    }
    if (!Is_Edge(v1, v2)) {
        throw std::invalid_argument("Edge does not exist");
    }
}

```

```

    cost[{v1, v2}] = new_cost;
}

```

9. Append_edge//adds an edge between two given vertices ;throws an exception if one or both vertices do not exist or if the edge already exists

Python source code:

```

def append_edge(self, x, y,c):
    if not self.valid_vertex(x):
        raise ValueError("Vertex {} does not exist.".format(x))
    if not self.valid_vertex(y):
        raise ValueError("Vertex {} does not exist.".format(y))
    if self.is_edge(x, y):
        raise ValueError("Edge already exists")
    self._out_edges[x].append(y)
    self._in_edges[y].append(x)
    self._costs[(x,y)] = c
    return True

```

C++ source code:

```

void Graph::Append_Edge(int x, int y, int c) {
    if (!Valid_Vertex(x) || !Valid_Vertex(y)) {
        throw std::invalid_argument("One or both vertices do not exist");
    }
    if (Is_Edge(x, y)) {
        throw std::invalid_argument("Edge already exists");
    }
    d_out[x].push_back(y);
    d_in[y].push_back(x);
    cost[{x, y}] = c;
}

```

10. Delete_edge//deletes an edge between two give vertices; throws an exception if one or both vertices do not exist of if the edge trying to be deleted does not exist

Python source code:

```

def delete_edge(self, x, y):
    if not self.valid_vertex(x):

```



```

        raise ValueError("Vertex {} does not exist.".format(x))
    if not self.valid_vertex(y):
        raise ValueError("Vertex {} does not exist.".format(y))
    if not self.is_edge(x, y):
        raise ValueError("Edge does not exist")
    self._costs.pop((x, y))
    self._out_edges[x].remove(y)
    self._in_edges[y].remove(x)

```

C++ source code:

```

void Graph::Delete_Edge(int x, int y) {
    if (!Valid_Vertex(x) || !Valid_Vertex(y)) {
        throw std::invalid_argument("One or both vertices do not exist");
    }
    if (!Is_Edge(x, y)) {
        throw std::invalid_argument("Edge does not exist");
    }
    cost.erase({x, y});
    d_out[x].erase(std::remove(d_out[x].begin(), d_out[x].end(), y), d_out[x].end());
    d_in[y].erase(std::remove(d_in[y].begin(), d_in[y].end(), x), d_in[y].end());
}

```

11.Append_vertex//adds a vertex to the graph; throws an exception if the graph already contains the give vertex

Python source code:

```

def append_vertex(self, x):
    if self.valid_vertex(x):
        raise ValueError("Vertex {} already exists.".format(x))
    self._out_edges[x] = []
    self._in_edges[x] = []

```

C++ source code:

```

void Graph::Append_Vertex(int x) {
    if (Valid_Vertex(x)) {
        throw std::invalid_argument("Vertex already exists");
    }
    d_in[x] = {};
    d_out[x] = {};
}

```

12. Delete_vertex//deletes a vertex form the graph(+all the edges that contain it);
throws exception if the vertex does not exist

Python source code:

```

def delete_vertex(self, x):
    if not self.valid_vertex(x):
        raise ValueError("Vertex {} does not exist.".format(x))
    for e in list(self.parse_out(x)): # Use list to avoid modifying while iterating
        self.delete_edge(x, e)
    for e in list(self.parse_in(x)):
        self.delete_edge(e, x)
    del self._in_edges[x]
    del self._out_edges[x]

```

C++ source code:

```

void Graph::Delete_Vertex(int x) {
    if (!Valid_Vertex(x)) {
        throw std::invalid_argument("Vertex does not exist");
    }
    // Remove all edges to/from this vertex
    for (int v : Parse_In(x)) {
        Delete_Edge(v, x);
    }
    for (int v : Parse_Out(x)) {
        Delete_Edge(x, v);
    }
    d_in.erase(x);
    d_out.erase(x);
}

```

13. Valid_vertex//return true if the vertex is in graph, false otherwise

Python source code:

```
def valid_vertex(self,v):
    return v in self._in_edges.keys()
```

C++ source code:

```
bool Graph::Valid_Vertex(int v) {
    return d_in.find(v) != d_in.end();
}
```

14.Str || Print_graph//prints the graph(for every vertex the list of inbound and outbound edges is shown)

Python source code:

```
def __str__(self):
    res = []
    res.append("Outbound:")
    for x in self.parse_vertices():
        res.append(f"{x}: " + " ".join(map(str, self.parse_out(x))))
    res.append("Inbound:")
    for x in self.parse_vertices():
        res.append(f"{x}: " + " ".join(map(str, self.parse_in(x))))
    return "\n".join(res)
```

C++ source code:

```
void Graph::Print_Graph() {
    std::cout << "Outbound:" << std::endl;
    for (const auto& pair : d_out) {
        std::cout << pair.first << ": ";
        for (int v : pair.second) {
            std::cout << v << " ";
        }
        std::cout << std::endl;
    }
}
```

```

std::cout << "Inbound:" << std::endl;
for (const auto& pair : d_in) {
    std::cout << pair.first << ": ";
    for (int v : pair.second) {
        std::cout << v << " ";
    }
    std::cout << std::endl;
}
}
std::map<int, std::vector<int>> Graph:: get_d_out() {
    return this->d_out;
}

```

15. Copy//returns a copy of the graph

Python source code:

```

def copy(self):
    in_copy = self._in_edges.copy()
    out_copy = self._out_edges.copy()
    costs_copy = self._costs.copy()
    return DirectedGraph(None, in_copy, out_copy, costs_copy)

```

C++ source code:

```

Graph Graph:: Copy_Graph() const {
    Graph new_graph;
    new_graph.d_out = this->d_out;
    new_graph.d_in=this->d_in;
    new_graph.cost=this->cost;
    return new_graph;
}

```

II. External functions

1.Read_graph_from_file//it returns a graph read from a text file. It s able to read 2 formats:

->format 1: the first line contains the number of vertices and the number of edges, then every line contains the edges with the associated cost.

->format 2: the first line is the word “special” so it can recognize the format, the next line contains all the vertices in the graph, then every line contains the edges with the associated cost.

Python source code:

```
def read_graph_from_file(filename):
    with open(filename, 'r') as f:
        lines = f.readlines()
        form = lines[0].strip()
        if form == 'special':
            l = lines[1].strip().split()
            li = []
            for i in l:
                i = int(i)
                li.append(i)
            g = DirectedGraph(li)
            for line in lines[2:]:
                line = line.strip()
                if line == '':
                    continue
                parts = line.split()
                vertex1 = int(parts[0])
                vertex2 = int(parts[1])
                cost = int(parts[2])
                g.append_edge(vertex1, vertex2, cost)
        else:
            l = lines[0].strip().split()
            g = DirectedGraph(int(l[0]))
            for line in lines[1:]:
                line = line.strip()
                if line == '':
                    continue
                parts = line.split()
                vertex1 = int(parts[0])
                vertex2 = int(parts[1])
                cost = int(parts[2])
```

```
        g.append_edge(vertex1, vertex2, cost)
    return g
```

C++ source code:

```
Graph read_graph_from_file(const std::string& filename) {
    std::ifstream file(filename);
    if (!file.is_open()) {
        throw std::ios_base::failure("Error opening file");
    }
    std::string line;
    std::getline(file, line);
    Graph g;
    if (line == "special") {
        std::getline(file, line);
        std::istringstream iss(line);
        std::vector<int> vertices;
        int v;
        while (iss >> v) {
            vertices.push_back(v);
        }
        g = Graph(vertices);
    } else {
        std::istringstream iss(line);
        int num_vertices;
        iss >> num_vertices;
        g = Graph(num_vertices);
    }
    while (std::getline(file, line)) {
        std::istringstream iss(line);
        int vertex1, vertex2, cost;
        if (!(iss >> vertex1 >> vertex2 >> cost)) {
            continue;
        }
        try {
            g.Append_Edge(vertex1, vertex2, cost);
        } catch (const std::invalid_argument& e) {
            std::cerr << "Error: " << e.what() << std::endl;
        }
    }
}
```

```

    }
}
file.close();
return g;
}

```

2. Write_graph_to_file//writes the graph in a text file

Python source code:

```

def write_graph_to_file(g,filename):
    with open(filename, "w") as f:
        f.write("special\n")
        vertices = ""
        for x in g.parse_vertices():
            vertices = vertices + f"{x} "
        f.write(vertices+"\n")
        for vertex1 in g.parse_vertices():
            for vertex2 in g.parse_out(vertex1):
                f.write(f"{vertex1}{vertex2}{g.get_cost(vertex1, vertex2)}\n")

```

C++ source code:

```

void write_graph_to_file(Graph& g,std::string& filename) {
    std::ofstream file(filename);
    if (!file.is_open()) {
        throw std::ios_base::failure("Error opening file");
    }
    file << "special\n";
    std::vector<int> vertices = g.Parse_Vertices();
    for (int x : vertices) {
        file << x << " ";
    }
    file << "\n";
    for (int vertex1 : vertices) {
        std::vector<int> out_vertices = g.Parse_Out(vertex1);
        for (int vertex2 : out_vertices) {
            file << vertex1 << " " << vertex2 << " " << g.Get_Cost(vertex1, vertex2) << "\n";
        }
    }
}

```

```

}
file.close();
}

```

3. Random_graph//generates and return a random graph with a give number of vertices and edges

Python source code:

```

def random_graph(n, m):
    if m>n**2:
        raise ValueError("m must be smaller than n^2")
    g = DirectedGraph(n)
    for i in range(m):
        x = randint(0, n - 1)
        y = randint(0, n - 1)
        c = randint(0, n - 1)
        while g.is_edge(x,y):
            x = randint(0, n - 1)
            y = randint(0, n - 1)
            c = randint(0, n - 1)
        g._out_edges[x].append(y)
        g._in_edges[y].append(x)
        g._costs[(x, y)] = c
    return g

```

C++ source code:

```

Graph random_graph(int num_vertices, int num_edges) {
    if (num_edges > num_vertices * num_vertices) {
        throw std::invalid_argument("Too many edges for the given number of vertices");
    }
    Graph g(num_vertices);
    std::srand(std::time(nullptr));
    int edges_added = 0;
    while (edges_added < num_edges) {
        int x = std::rand() % num_vertices;
        int y = std::rand() % num_vertices;
        if (x != y && !g.Is_Edge(x, y)) {

```



```
int cost = std::rand() % 100;
try {
    g.Append_Edge(x, y, cost);
    edges_added++;
} catch (const std::invalid_argument& e) {
    std::cout << e.what() << std::endl;
}
}
}
return g;
}
```

III. Main

For testing the methods implemented in the DirectedGraph class, I implemented a menu containing all the options.

